UNIVERSITY OF AUCKLAND
Waipapa Taumata Rau
NEW ZEALAND

SCIENCE

COMPSCI 714
Lectutorial 4: Learning with sequences – RNN & Transformer

Working with sequential data & RNNs

The Transformer architecture and how does it addresses some of the RNN limitations

Instructor: Thomas Lacombe

Learning outcomes

- **Explain** the motivation of self-attention and Transformer models.
- **Break down the components** of the Transformer architecture, including self-attention, multi-head attention, positional encoding, encoder–decoder structure, and masking.
- **Explain** how the Transformer architecture addresses limitations of RNNs.
- **Apply** sequence models in practice, using PyTorch to experiment with RNNs, LSTM/GRU models, and Transformer-based architectures.

Extra resources

If you want to go further:

- *Hands-on Machine Learning with Scikit-Learn and PyTorch*, Géron A.
  - Chapters 14, 15, 16 and 17.

Coding time 1: Time series and RNNs

Transformer - Motivation

- Traditional RNNs suffer some drawbacks

  1. Sequential processing leading to slow training times (especially for long sequences) + hard to parallelise.

  2. Difficulty to model long term dependencies as information from earlier steps becomes increasingly diluted.

- The Transformer architecture addresses these issues by using the self-attention mechanism.

Example: Translation task

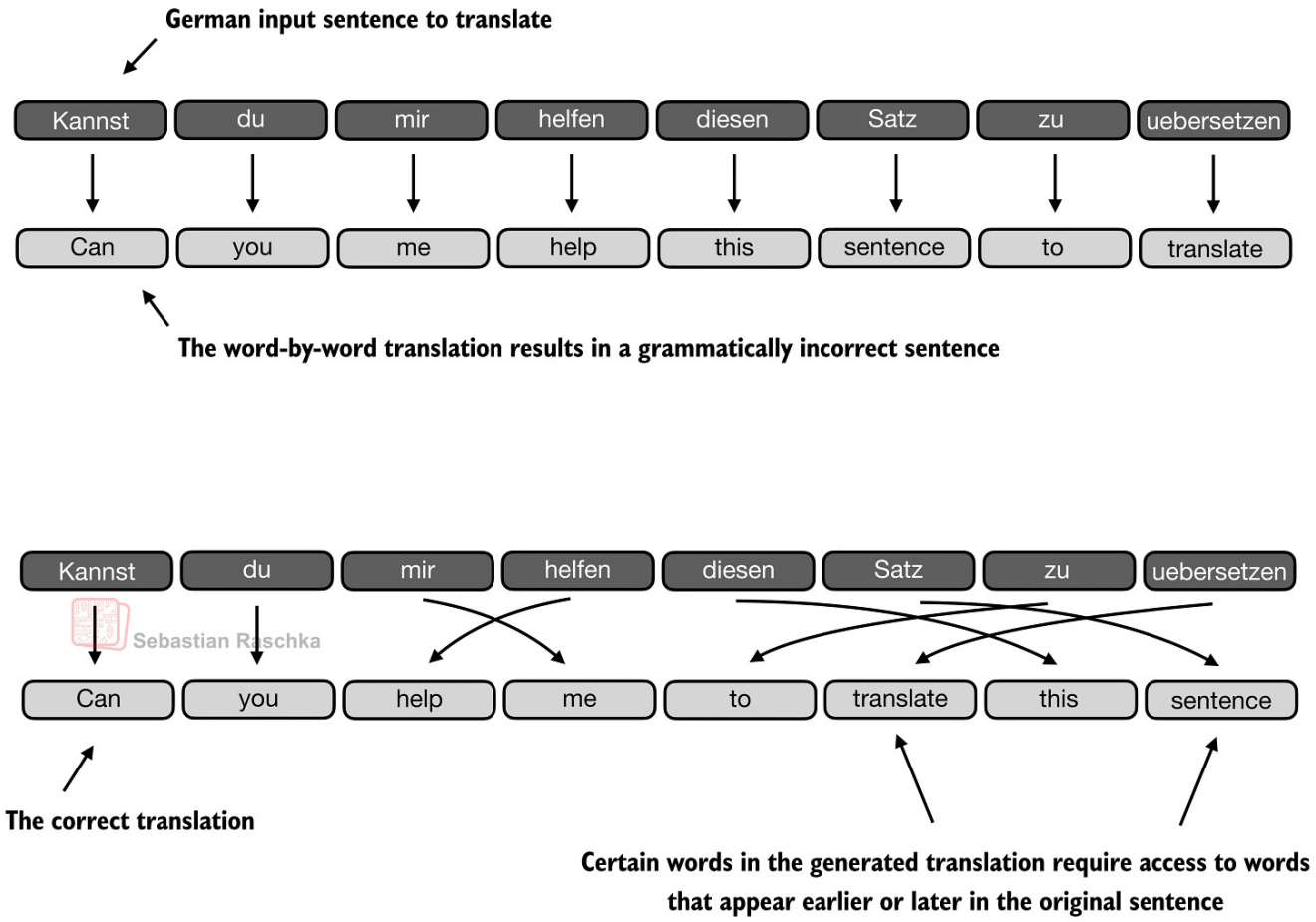


Image source *LLMs-from-scratch*, S. Raschka

Example: Translation task

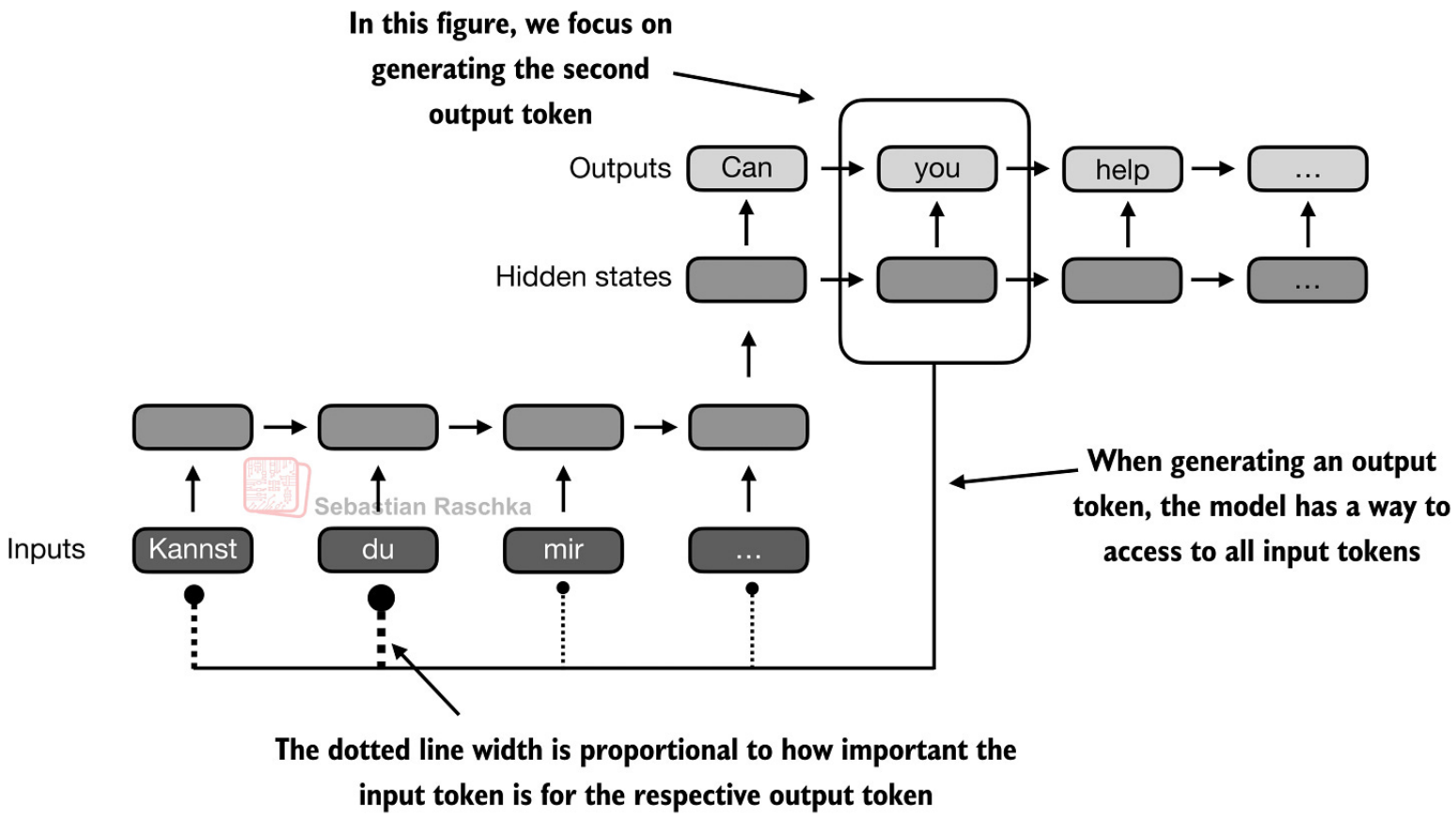


Image source *LLMs-from-scratch*, S. Raschka

Self-attention mechanism

Self-attention allows a model to weigh the importance of different parts of the input sequence (i.e., token) when making predictions.

It produces a **refined representation** of the tokens, incorporating relevant context, to be more useful to the task.

E.g., with language models, the model "focuses" on different words of the sequence depending on their relevance to the task at hand:

Predicting the next word in: "The rabbits are eating"

What are the important words to focus on for the prediction?

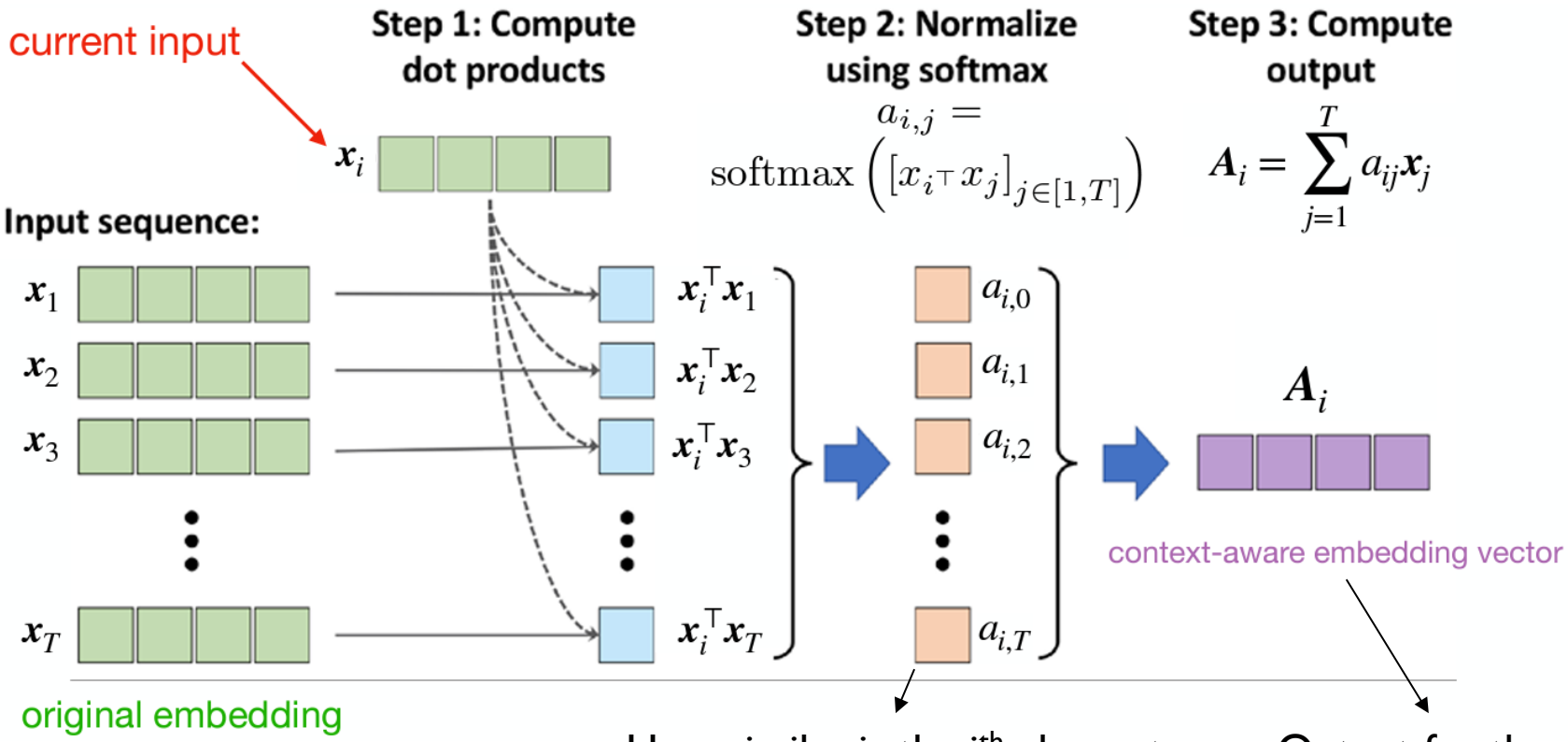
Representing tokens with embeddings

Individual tokens (e.g., words) are represented as vector of numerical values in a lower dimension space. Such representations are called embeddings.

Embeddings aim at capturing the meaning of tokens and their relationship to other tokens (e.g., semantic similarity, syntax relationships).

- Bag of words (BOW) are one of the simplest word embedding, but they can be intensive to compute and they fail to capture the relationship between words (i.e., do not consider the order).
- Modern word embeddings are learned through ML and take in considering the local or global context of the words. A few popular embeddings: Word2Vec, GloVe, FastText and ElMo.

Self-attention – basic version

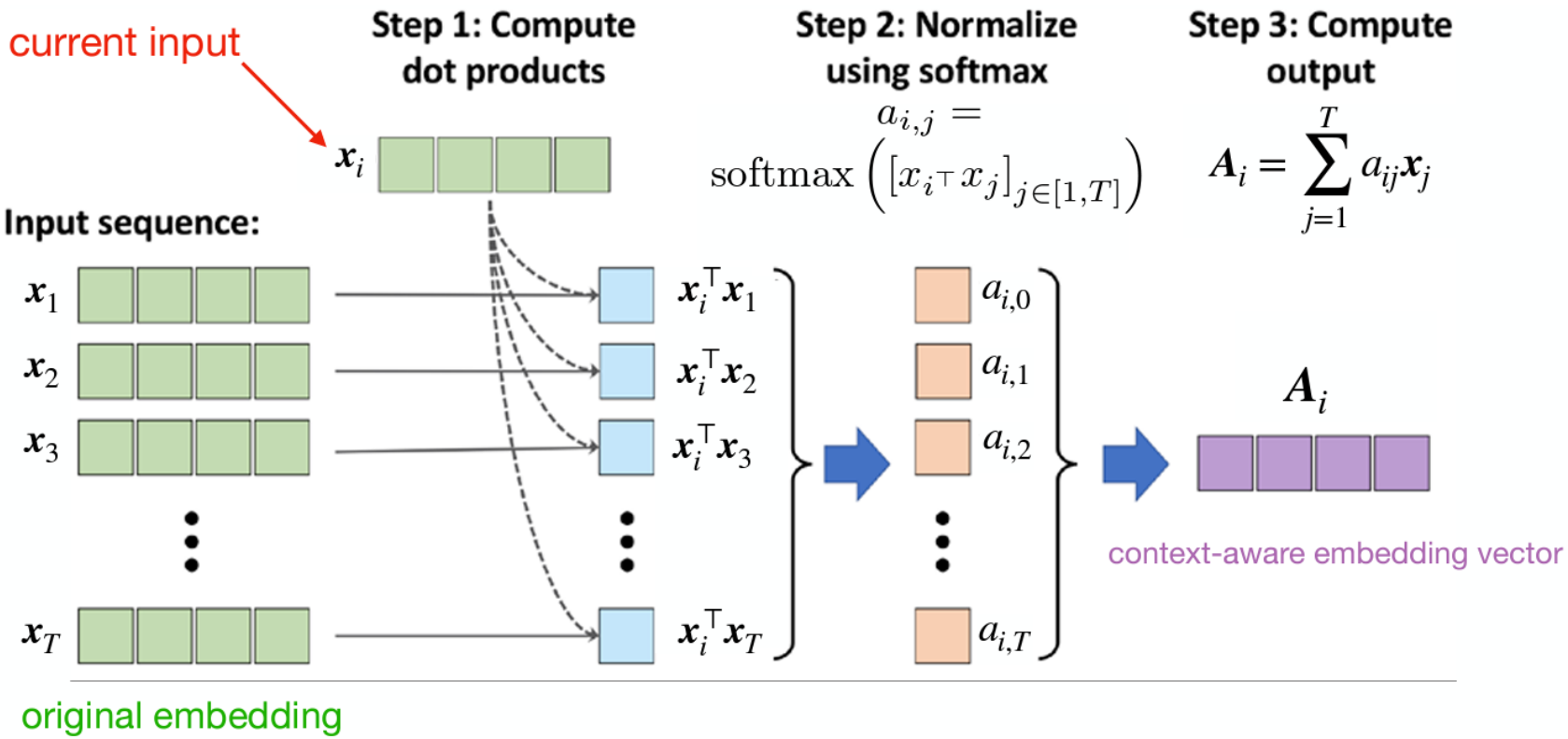


How similar is the ith element to the other elements of the sequence?

Output for the ith element

Image source: Raschka & Mirjalili 2019. Python Machine Learning, 3rd edition

Self-attention – basic version



Limitation of this basic version: no learnable parameters!

Image source: Raschka & Mirjalili 2019. Python Machine Learning, 3rd edition

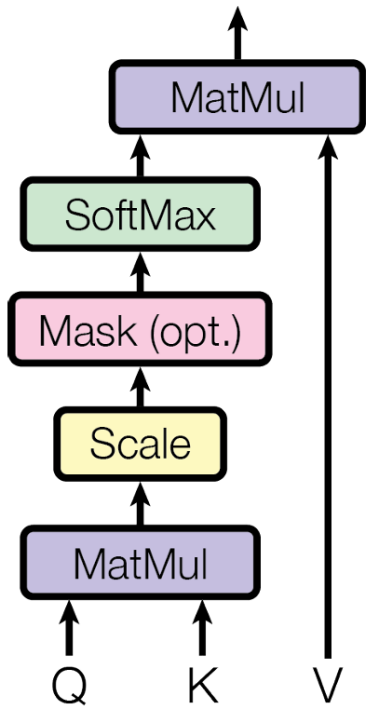
Scaled Dot-Product attention

The Transformer architecture uses an implementation of self-attention called « Scaled Dot-Product attention ».

1. Transform the input matrix $X = [x_1, x_2, \dots, x_n]$ into 3 matrices, i.e., views:
   - Query: $Q = W_q X$
   - Key: $K = W_k X$
   - Value: $V = W_v X$

W_q, W_k, W_v are learnable weight matrices that transform the input matrix into query, key and value.

Scaled Dot-Product Attention



[Attention Is All You Need, Vaswani et al., 2017]

Scaled Dot-Product attention

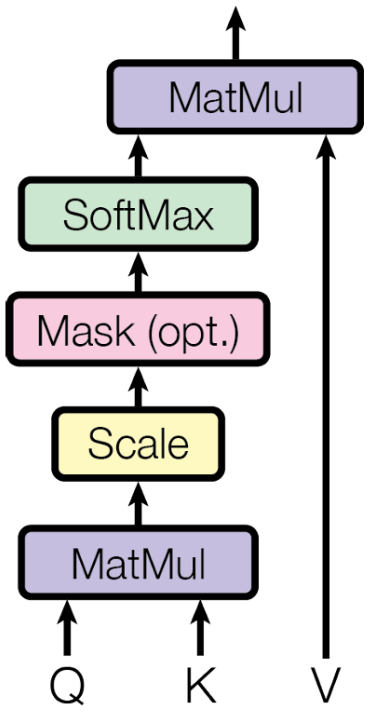
2. Calculate the attention « score »:

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})\mathrm{V}$$

d_k is the size of the keys and queries
(d_k = d_q= d_v in the original paper).

Scaling prevents the dot products to grow large, thus avoiding vanishing gradients (application of softmax to large values would yield small gradients).

Scaled Dot-Product Attention



[Attention Is All You Need, Vaswani et al., 2017]

Scaled Dot-Product attention

- Query: $Q = W_q X$
  - What contextual information is important to a token to improve its representation w.r.t. the task and context
- Key: $K = W_k X$
  - What aspects of a token are relevant for being matched by a query
- Value: $V = W_v X$
  - What is the actual information passed along by a token if it is attended to
- QK^T: Token-to-token relevance scores
- A: Attention weights matrix
- Z: Output matrix, i.e., attention-aware embeddings for each token

- Attention: Use Q to find relevant K and pull their V

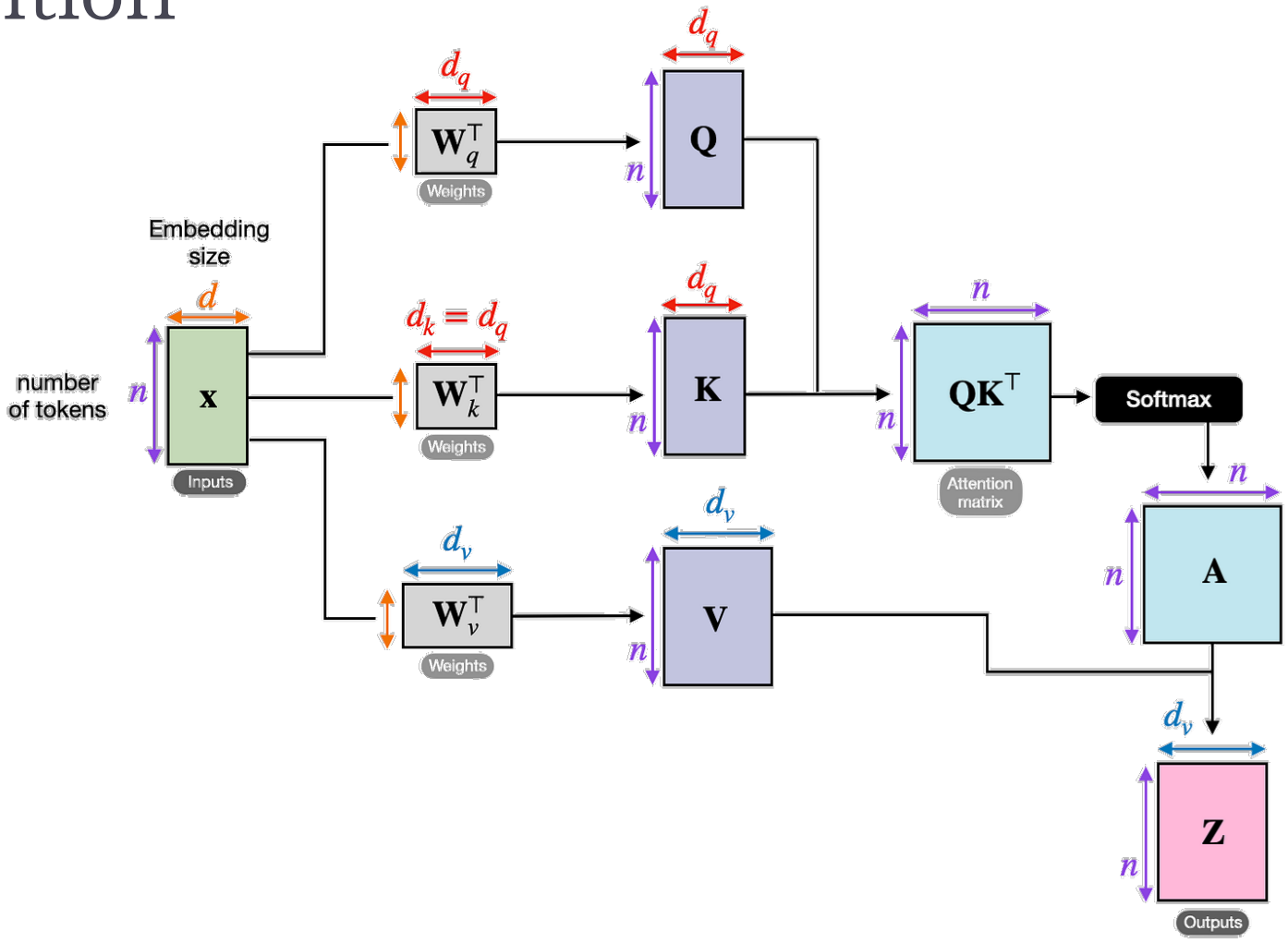


Image source *Understanding and Coding Self-Attention*, *S. Raschka*

Scaled Dot-Product attention

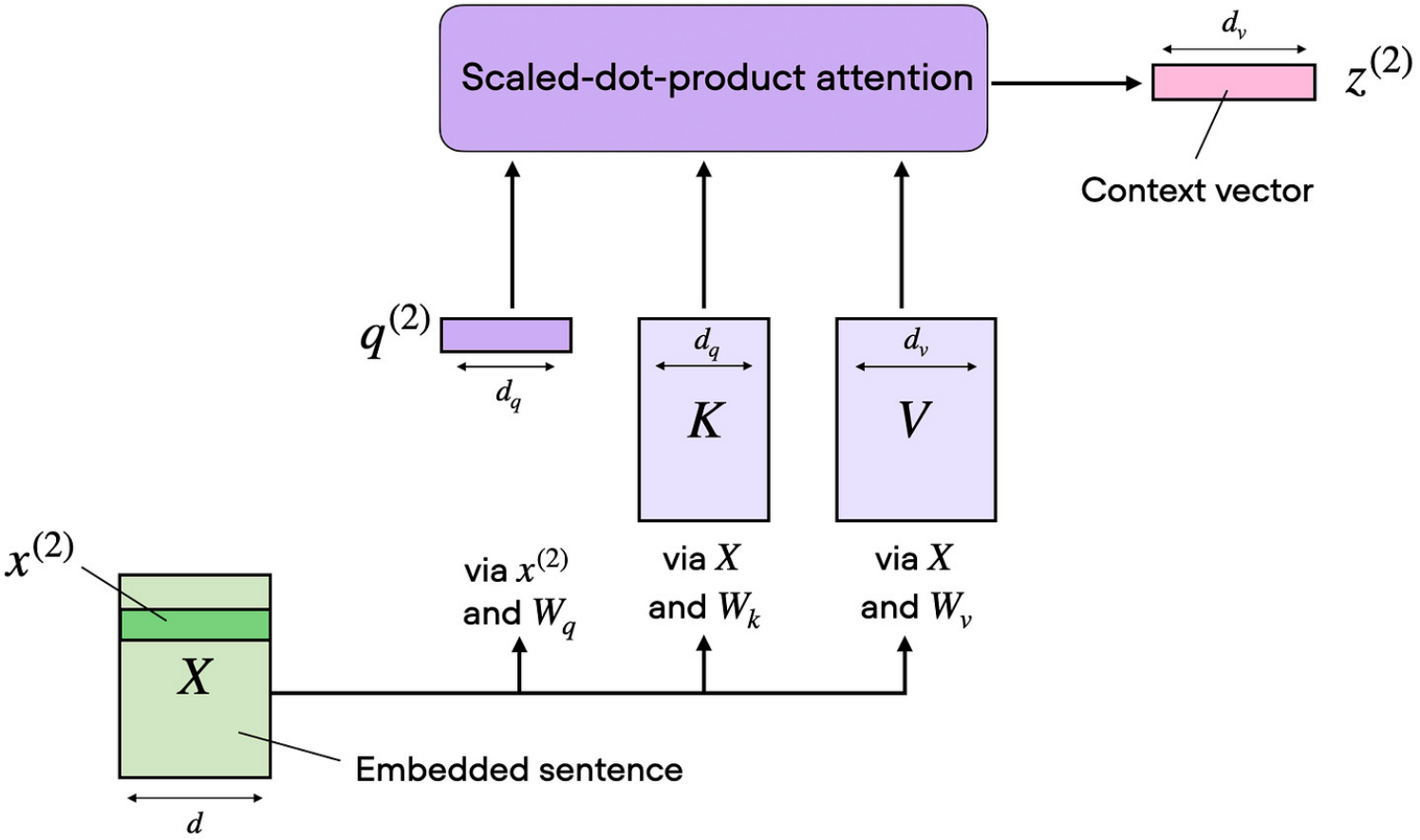
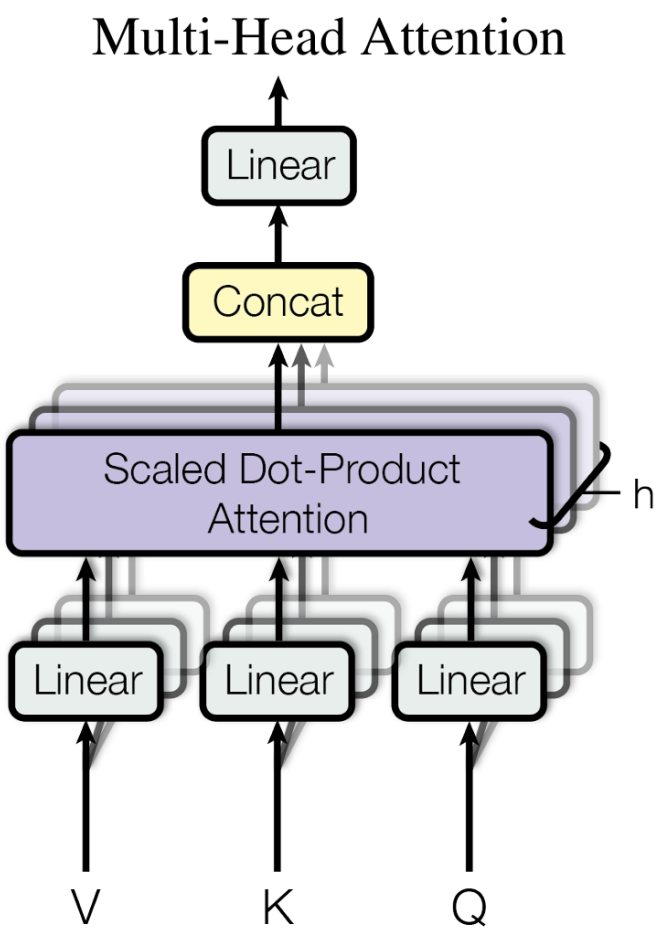


Image source *Understanding and Coding Self-Attention, S. Raschka*

Multi-Head attention

The Transformer architecture actually stacks several scaled dot-product attention layers in parallel.

- *h* parallel layers, also called “heads”.
- Values, keys and queries are projected linearly *h* times with different learned linear projections.
- Each projection is the input of an head, which has its own different W_q, W_k, W_v matrices.



[Attention Is All You Need, Vaswani et al., 2017]

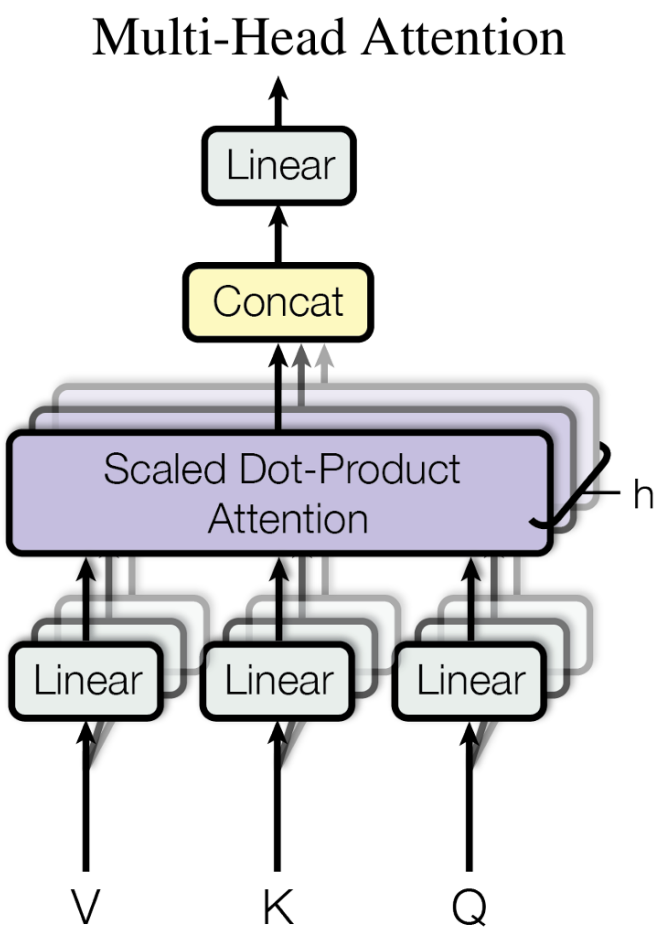
Multi-Head attention

The Transformer architecture actually stacks several scaled dot-product attention layers in parallel.

- Results are concatenated and projected again to obtain the output:

$$\mathrm{MultiHead}(\mathrm{Q},\mathrm{K},\mathrm{V}) = \mathrm{Concat}(\mathrm{head}_1, \dots, \mathrm{head}_h)\, W_o$$

$$\text{where } \mathrm{head}_i = \mathrm{Attention}(QW_{q,i}, KW_{k,i}, VW_{v,i})$$
$$\equiv \mathrm{Attention}(XW_{q,i}, XW_{k,i}, XW_{v,i})$$



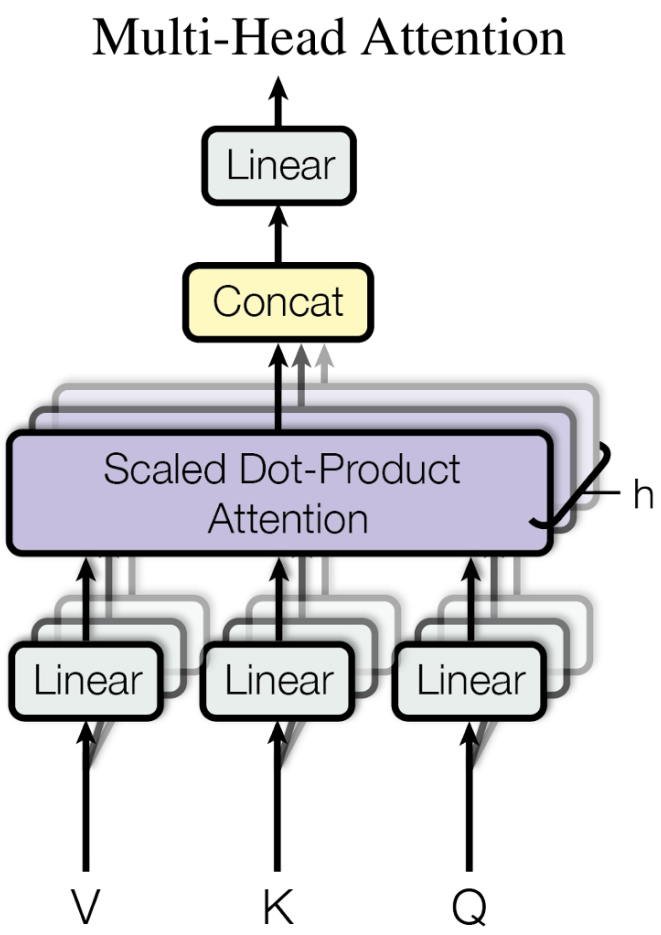
[Attention Is All You Need, Vaswani et al., 2017]

Multi-Head attention

Multi-head attention is beneficial because it allows the Transformer to focus on different aspects of the input sequence.

Using a single attention head would have an averaging effect.

Intuitively, the multi-head attention allows the Transformer to attend to different types of relationships, i.e., different aspects of the sequence in parallel.



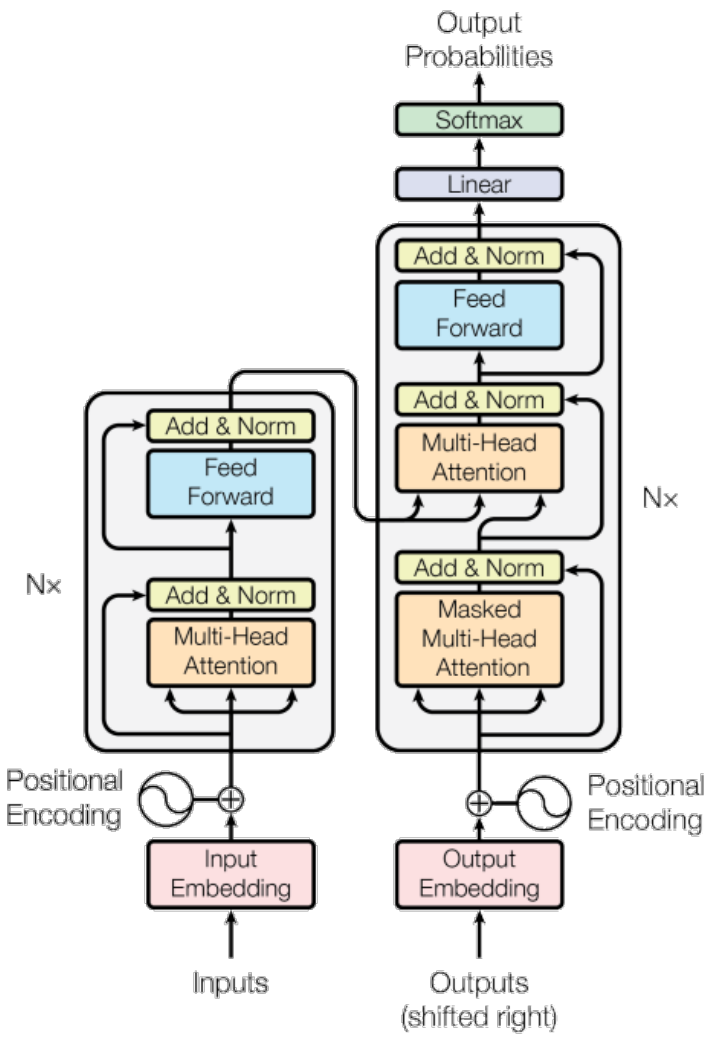
[Attention Is All You Need, Vaswani et al., 2017]

Transformer – Full architecture

- Seq2Seq architecture (encoder/decoder)
  - Encoder: takes an input sequence and produces a set of hidden representations, also known as context vector.
  - Decoder: takes the context vector and generates the output sequence.

- Not an RNN!
  - No sequential processing, it uses embedded representations to **encode positions in the sequence**.

- Uses the **attention mechanism** to capture which parts of the sequence are most important for each token.

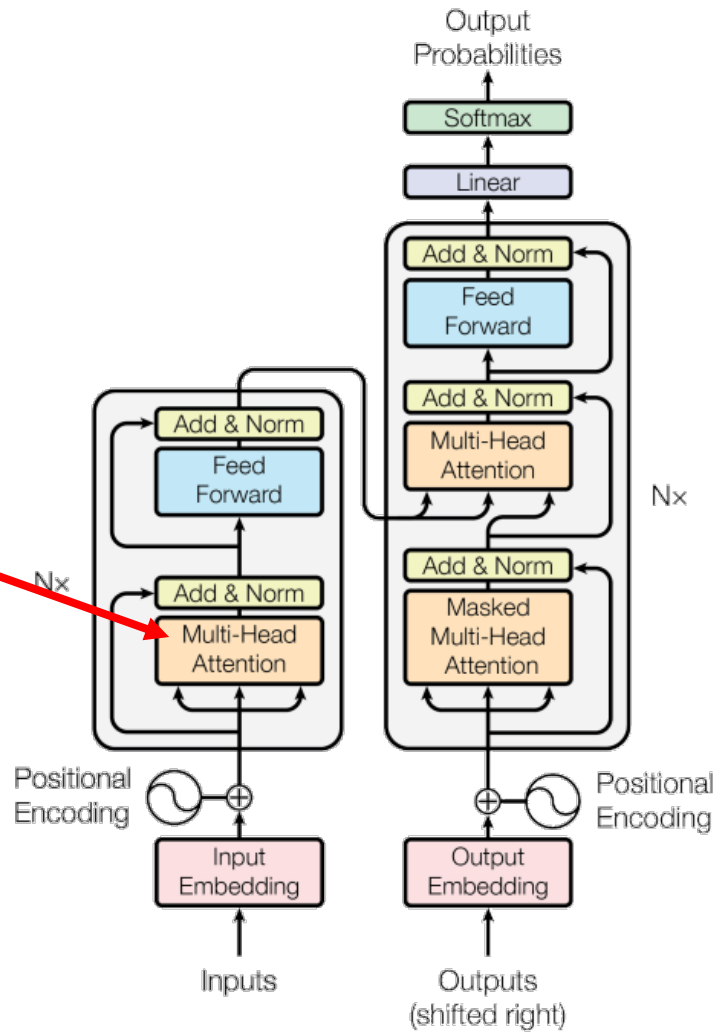


[Attention Is All You Need, Vaswani 2017]

Transformer – Full architecture

Encoder block components:

1. Multi-head attention

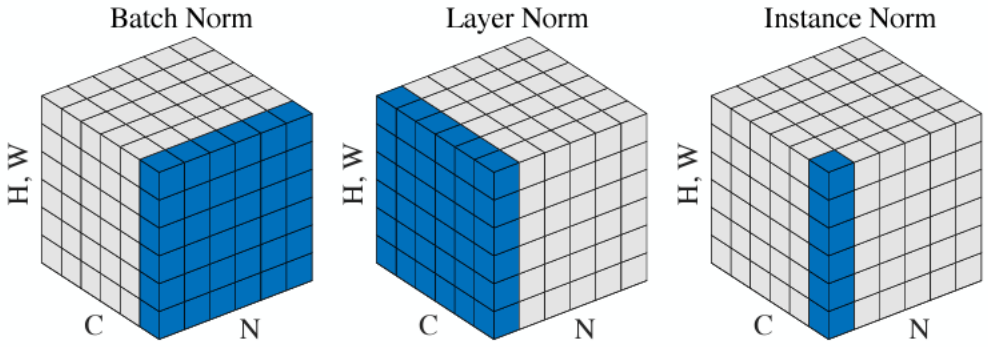


[Attention Is All You Need, Vaswani 2017]

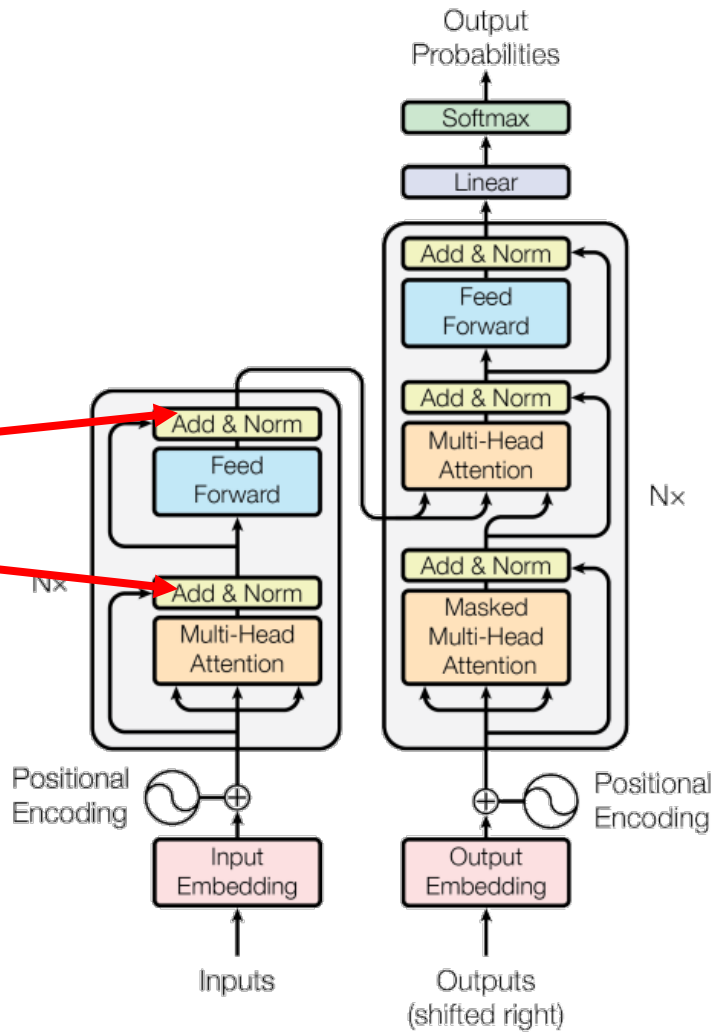
Transformer – Full architecture

Encoder block components:

1. Multi-head attention
2. Layer normalisation



[Group normalization. ECCV, Wu, Y., & He, K., 2018]



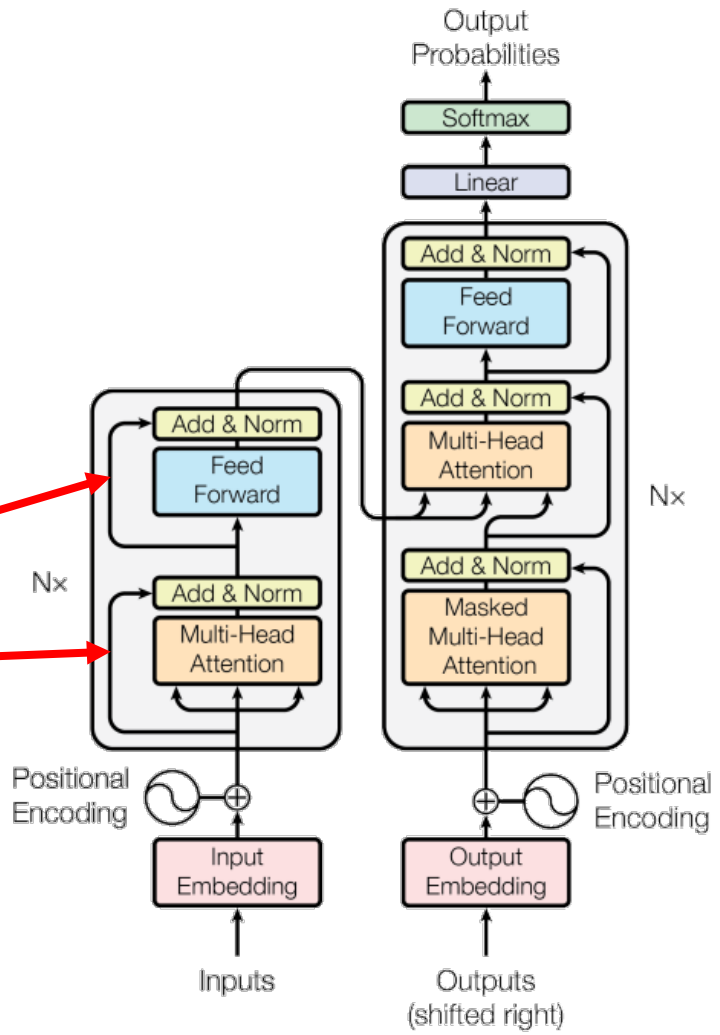
[Attention Is All You Need, Vaswani 2017]

Transformer – Full architecture

Encoder block components:

1. Multi-head attention
2. Layer normalisation
3. Residual/skip connection

$$\text{out}_{\text{norm}} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

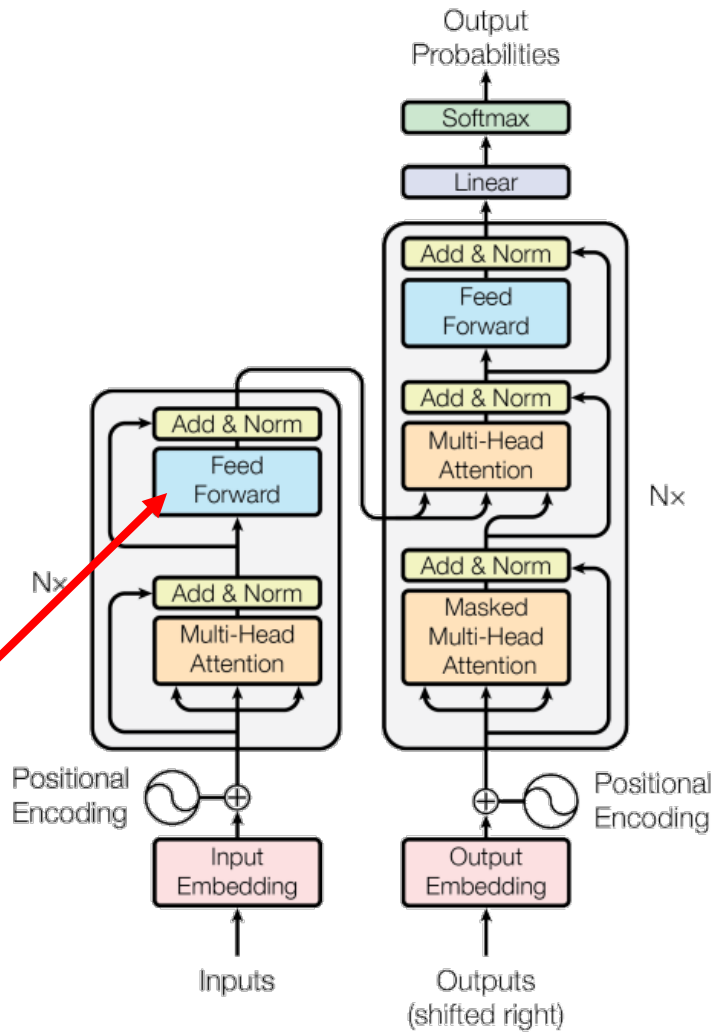


[Attention Is All You Need, Vaswani 2017]

Transformer – Full architecture

Encoder block components:

1. Multi-head attention
2. Layer normalisation
3. Residual/skip connection
4. Fully-connected feed forward NN (multilayer perceptron)
   - Transforms each token representation to adapt it for the next block → can be easily parallelised (treat all tokens in parallel).

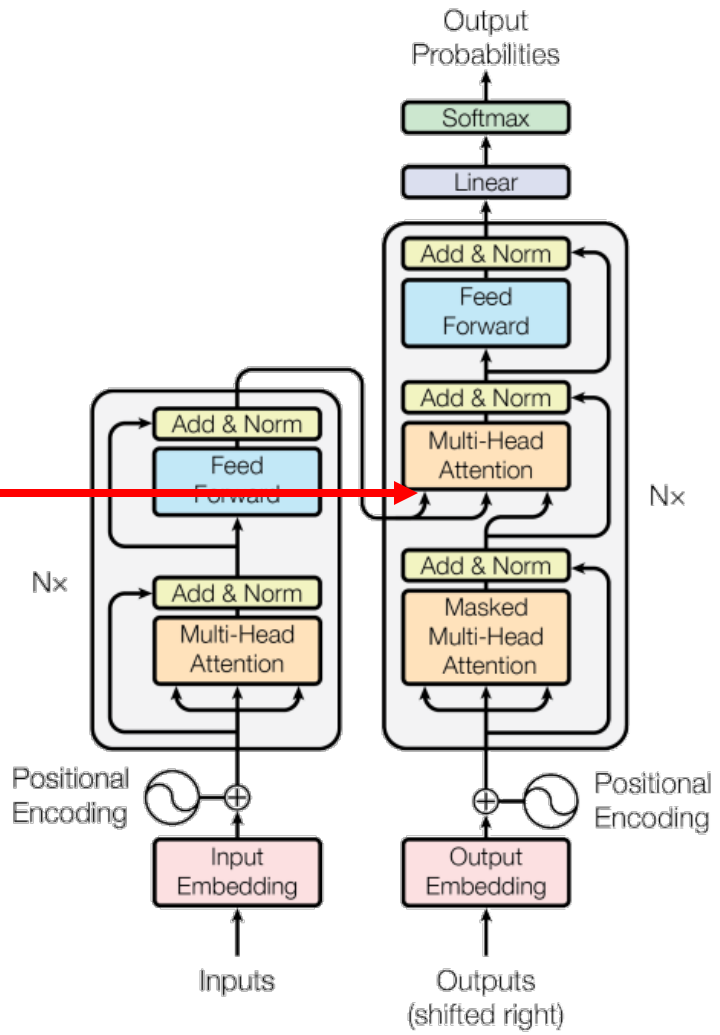


[Attention Is All You Need, Vaswani 2017]

Transformer – Full architecture

Decoder block components:

- Similar components as the encoder.
- Encoder output passed to multi-head attention.

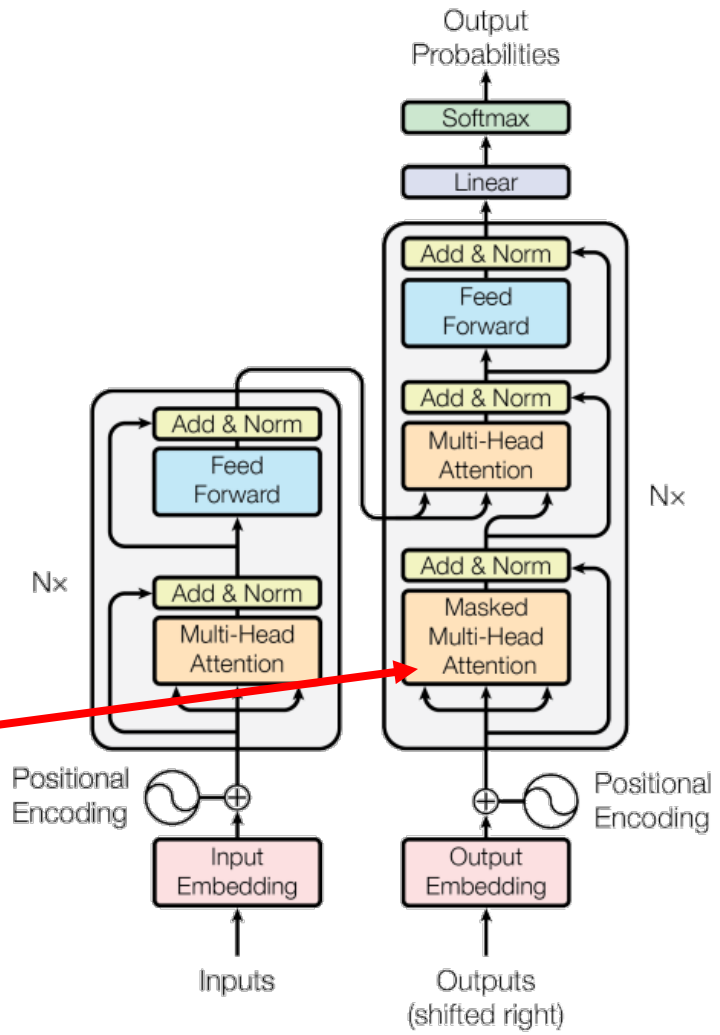


[Attention Is All You Need, Vaswani 2017]

Transformer – Full architecture

Decoder block components:

- Similar components as the encoder.
- Encoder output passed to multi-head attention.
- First multi-head attention sublayer is masked to prevent the model to "cheat" and look at what is coming next in the sentence at training time.
  - Ensures the prediction at position i only depends on outputs at position less than i.



[Attention Is All You Need, Vaswani 2017]

Transformer – Full architecture

Several encoder and decoder blocks are stacked.

- In original Transformer paper: $N_x = 6$

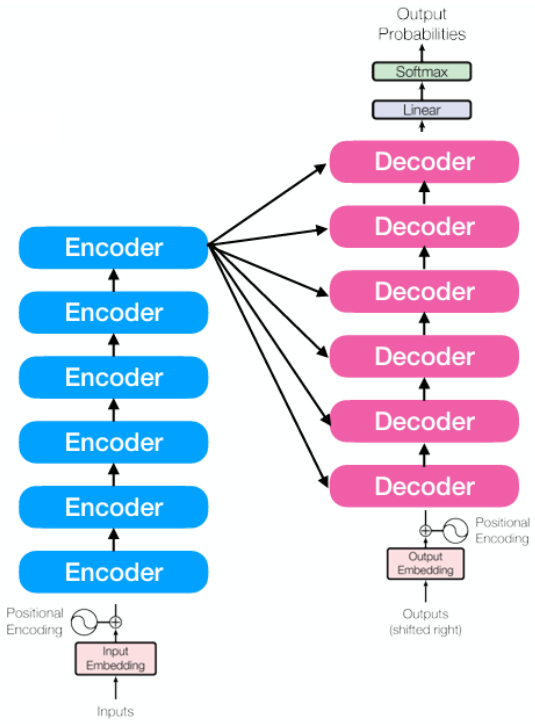
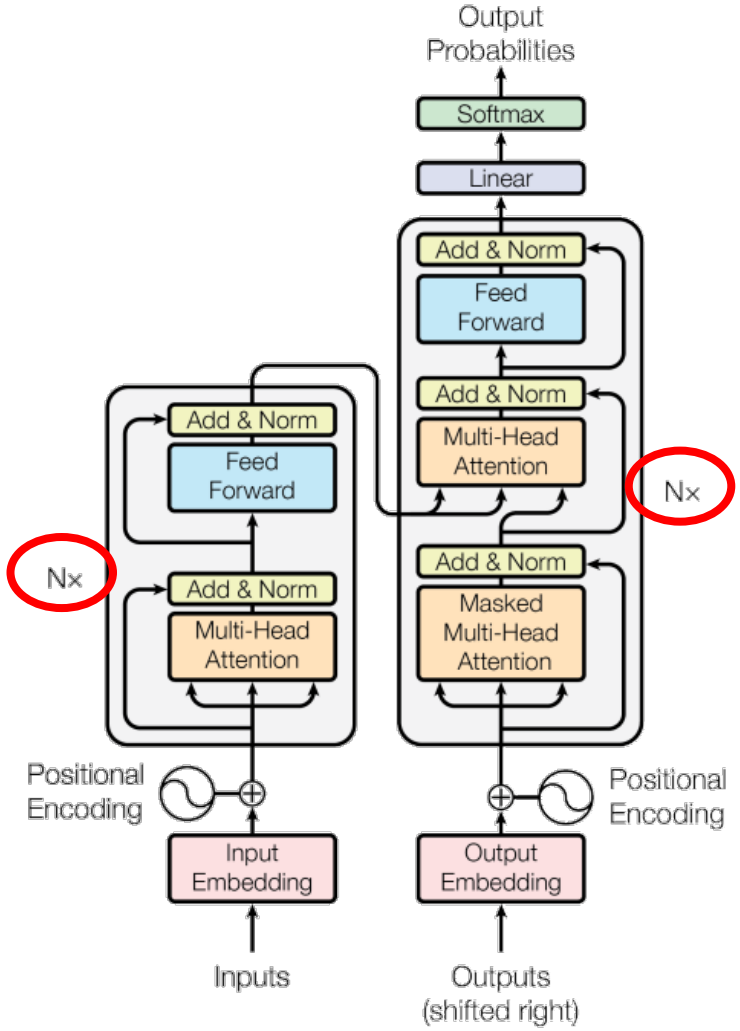


Image source: Sebastian Raschka, STAT 453: Intro to Deep Learning



[Attention Is All You Need, Vaswani 2017]

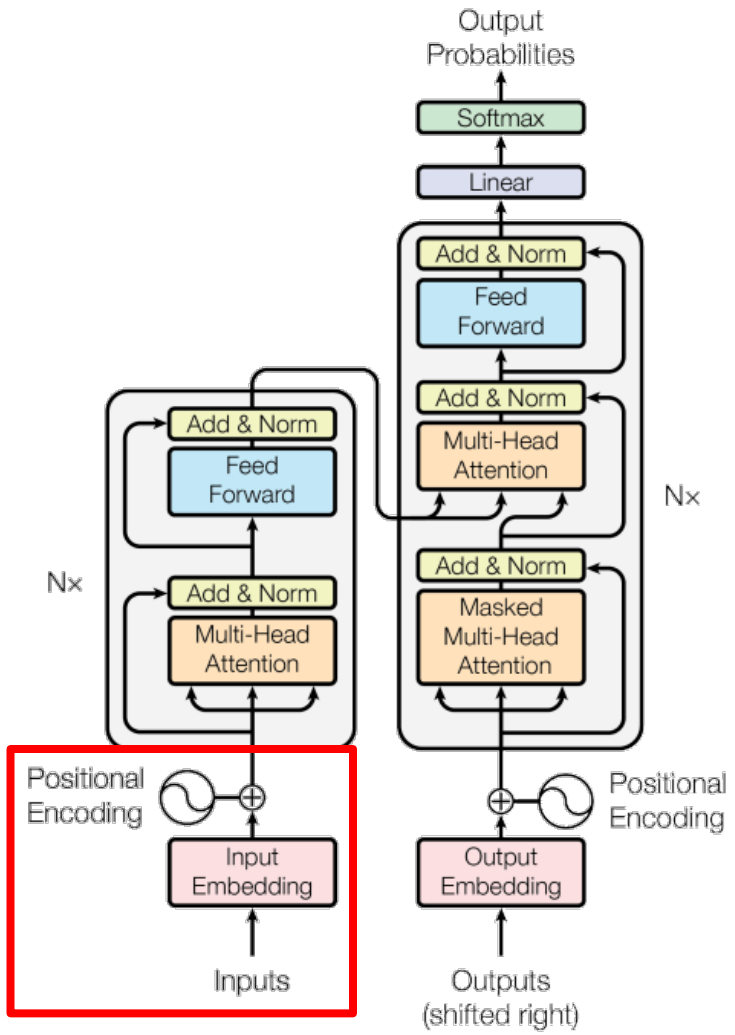
Transformer – Full architecture

Input/output embedding:

- Original embeddings size: $d_{model} = 512$
- Embeddings are learned during the training process.

Positional encoding:

- RNNs were by design learning about the position of elements in the sequence.
- Transformers lose this information as they do not process the data sequentially!
- Positional encoding is used to retrieve the order information.

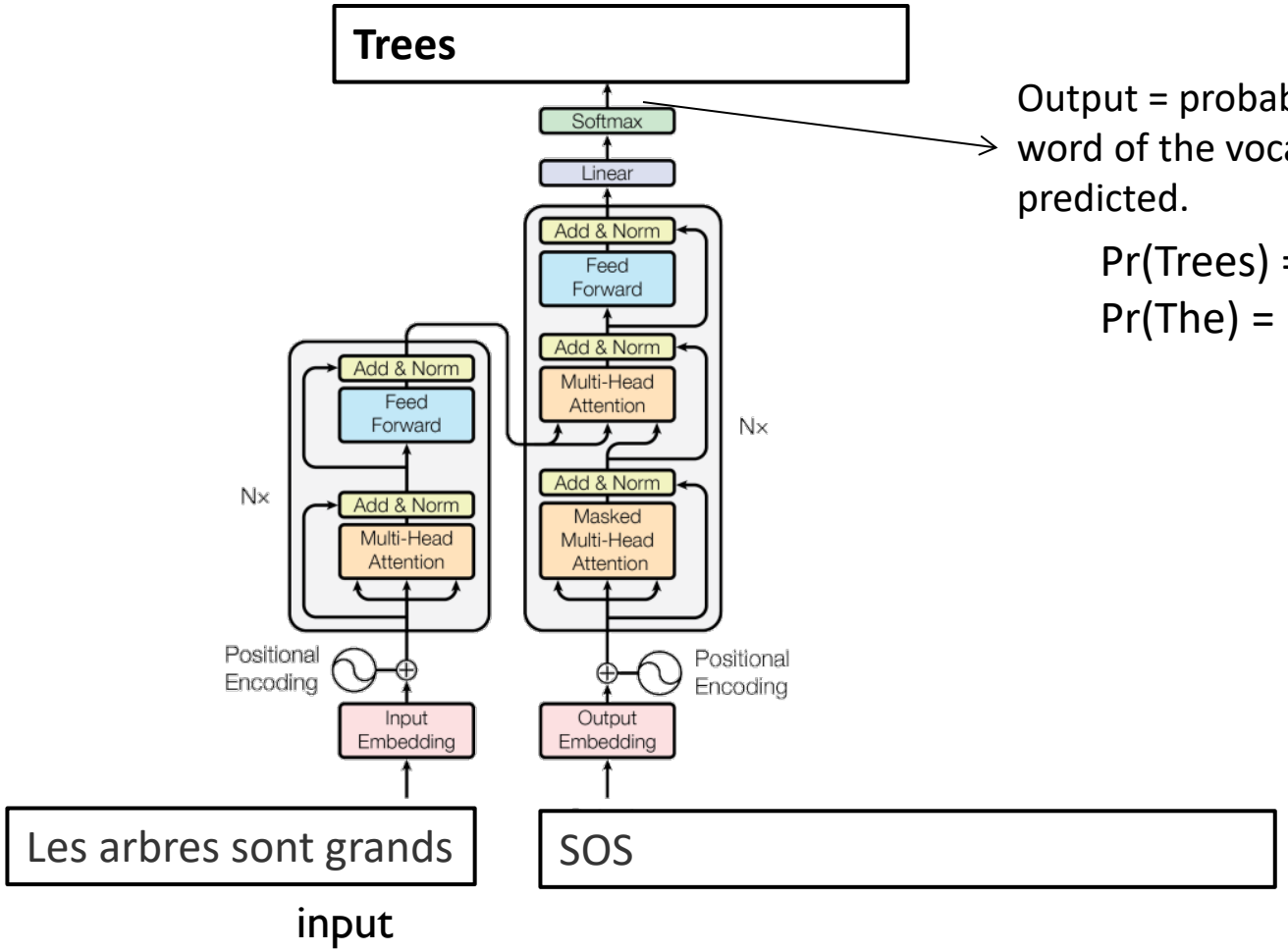


[Attention Is All You Need, Vaswani 2017]

Full explanation with example for positional embedding (Hedu AI - Youtube)

Transformer – Translation

E.g., predicted word = highest softmax probability.



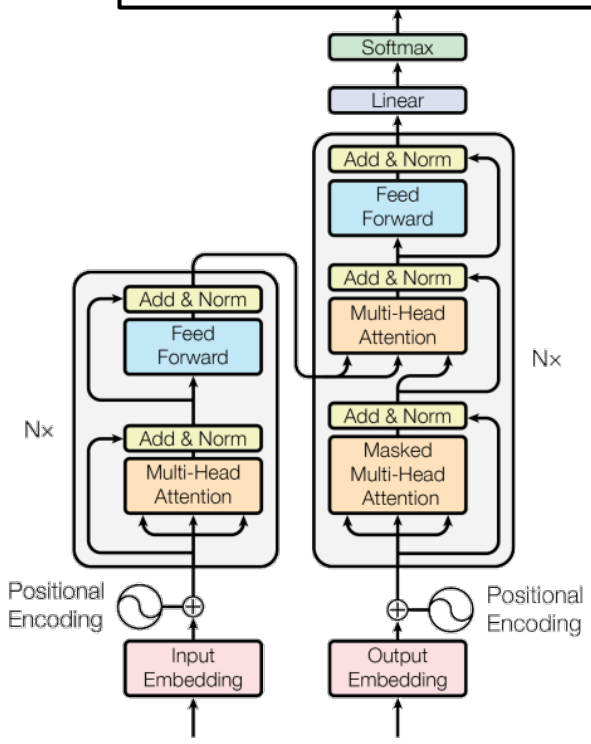
Output = probabilities of each word of the vocabulary to be predicted.

Pr(Trees) = 0.8
Pr(The) = 0.2

Transformer – Translation

Trees **are**

Pr(are) = 0.95
Pr(is) = 0.05

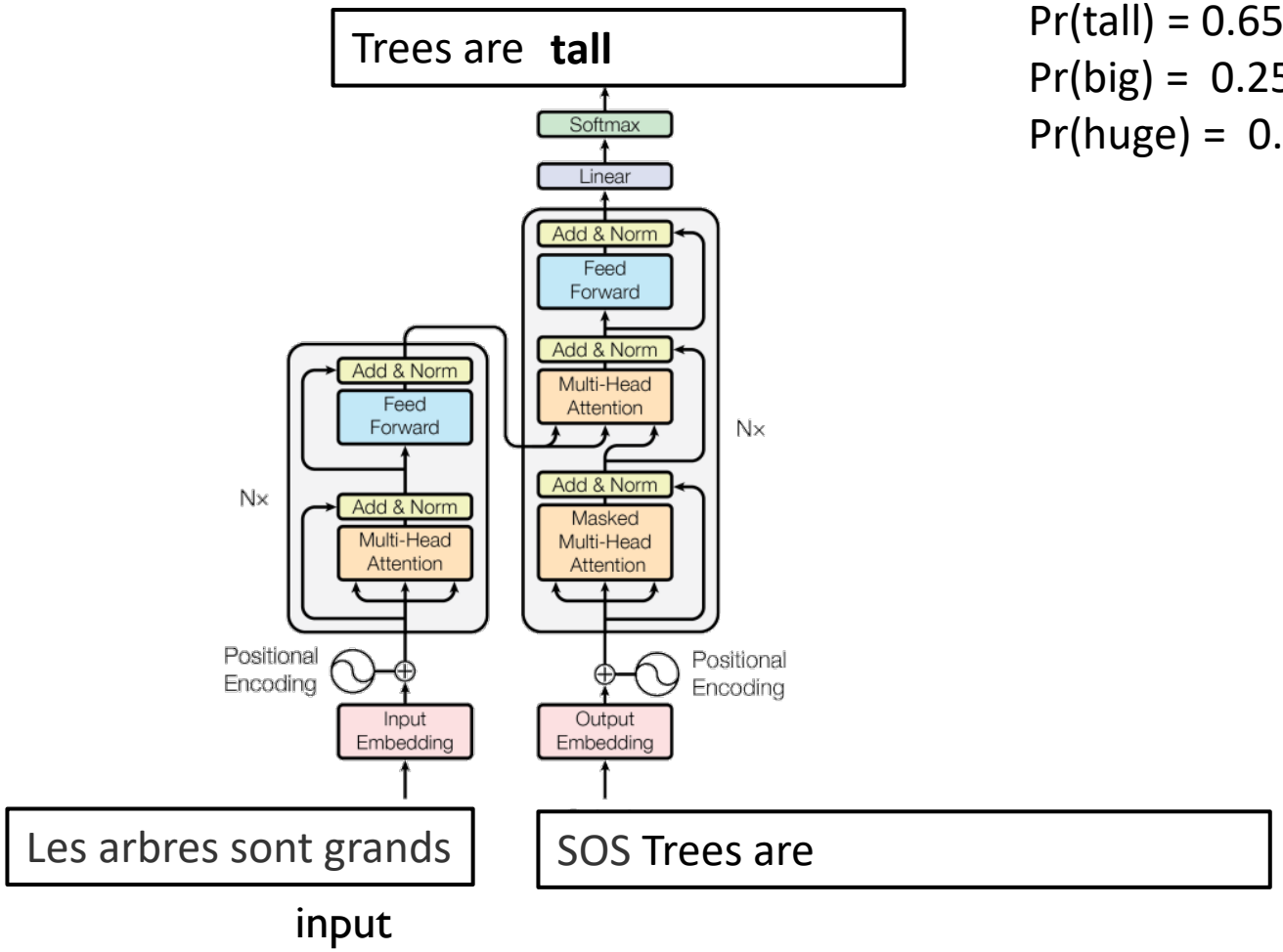


Les arbres sont grands

input

SOS Trees

Transformer – Translation



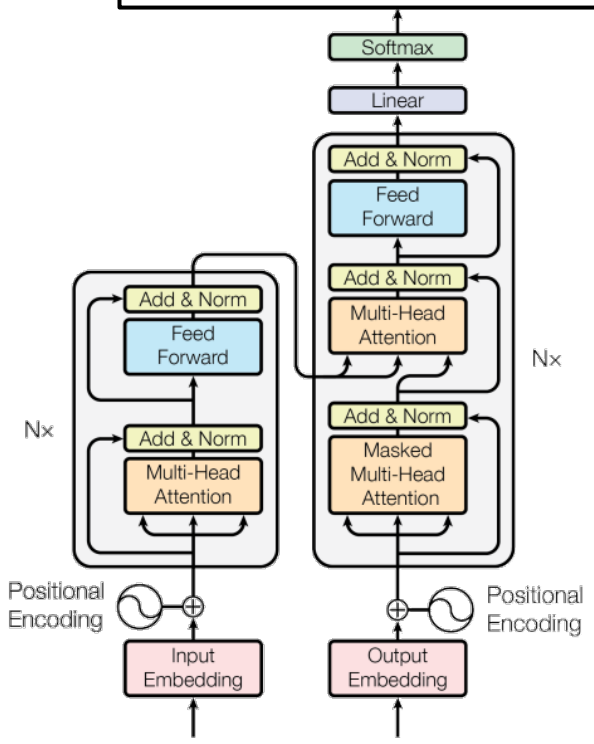
Pr(tall) = 0.65
Pr(big) = 0.25
Pr(huge) = 0.1

Transformer – Translation

Trees are tall **ESO**

Pr(EOS) = 0.99

…

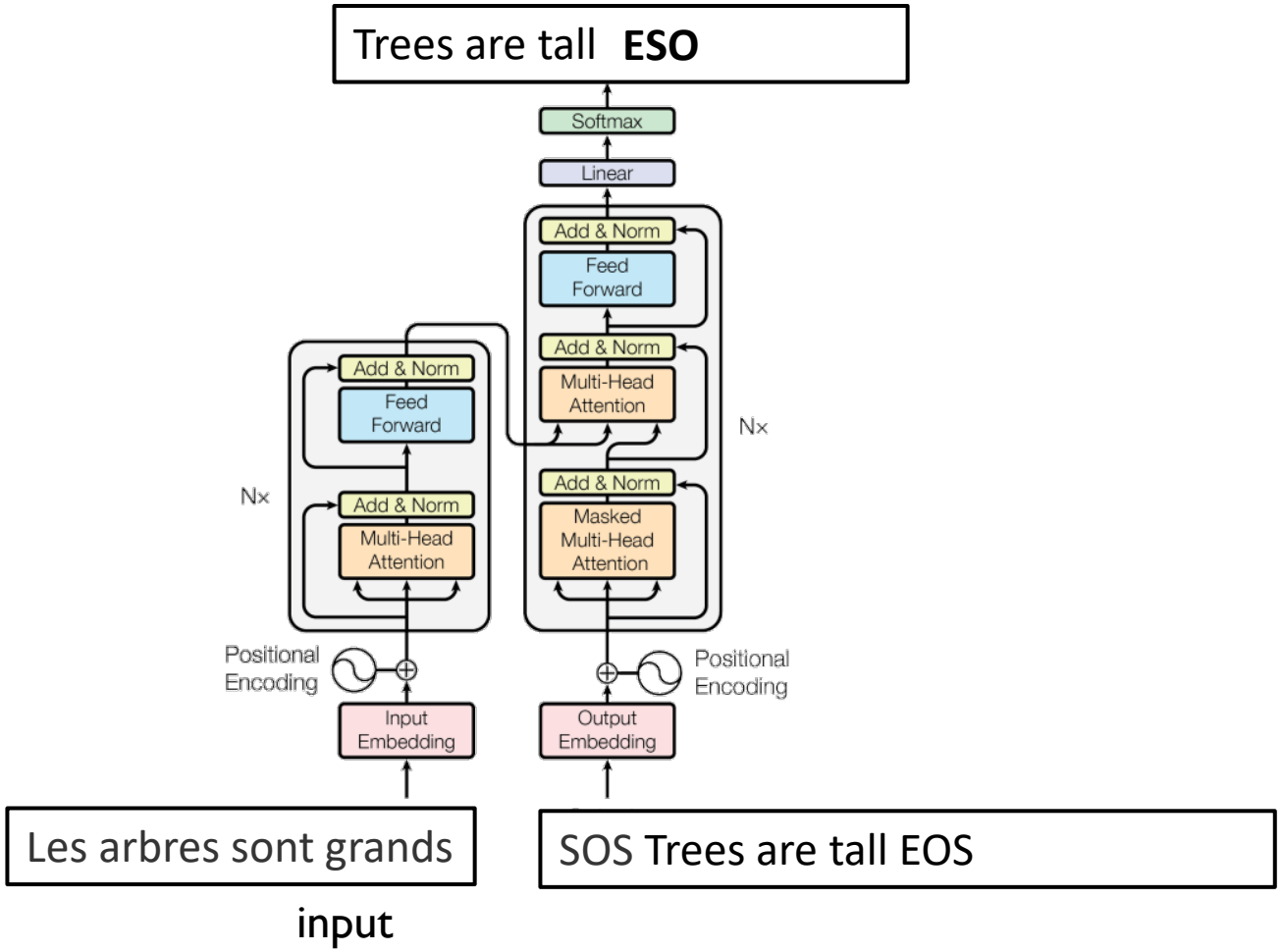


Les arbres sont grands

input

SOS Trees are tall

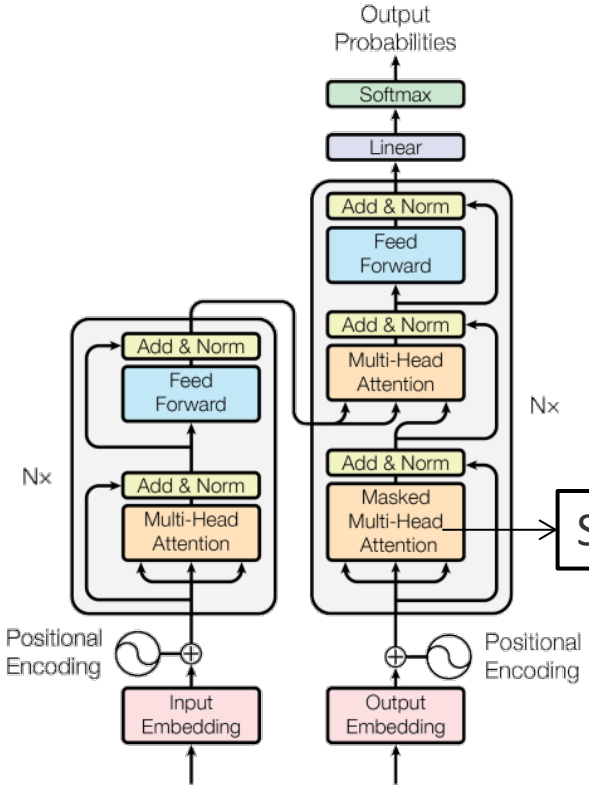
Transformer – Translation



Transformer – Translation training

Target: Trees are tall EOS

Loss is calculated between predicted and target words at each position (cross entropy loss).



At training time the transformer computes all next words probabilities in parallel.
It outputs 1 probability distribution per token position.
Masking is used to avoid « cheating ».

SOS Trees [redacted]

Les arbres sont grands

input

SOS Trees are tall EOS

whole output (**only when training**)

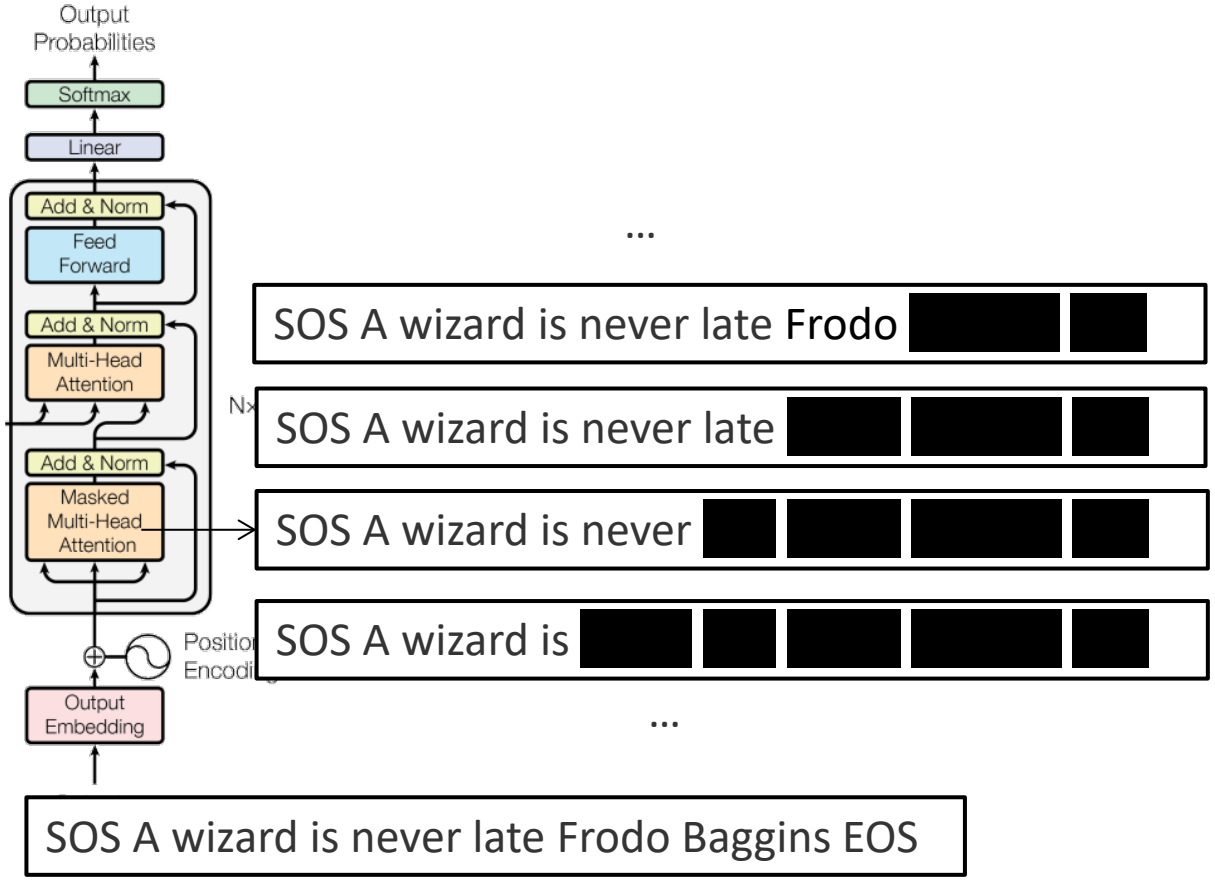
Transformer – Next word prediction

Target | A wizard is never late Frodo Baggins EOS

Only uses the decoder!

No need for context vector (not a source-target task).

Decoder learns its own output.



whole output (**only when training**)

Transformers – Attention Visualisation

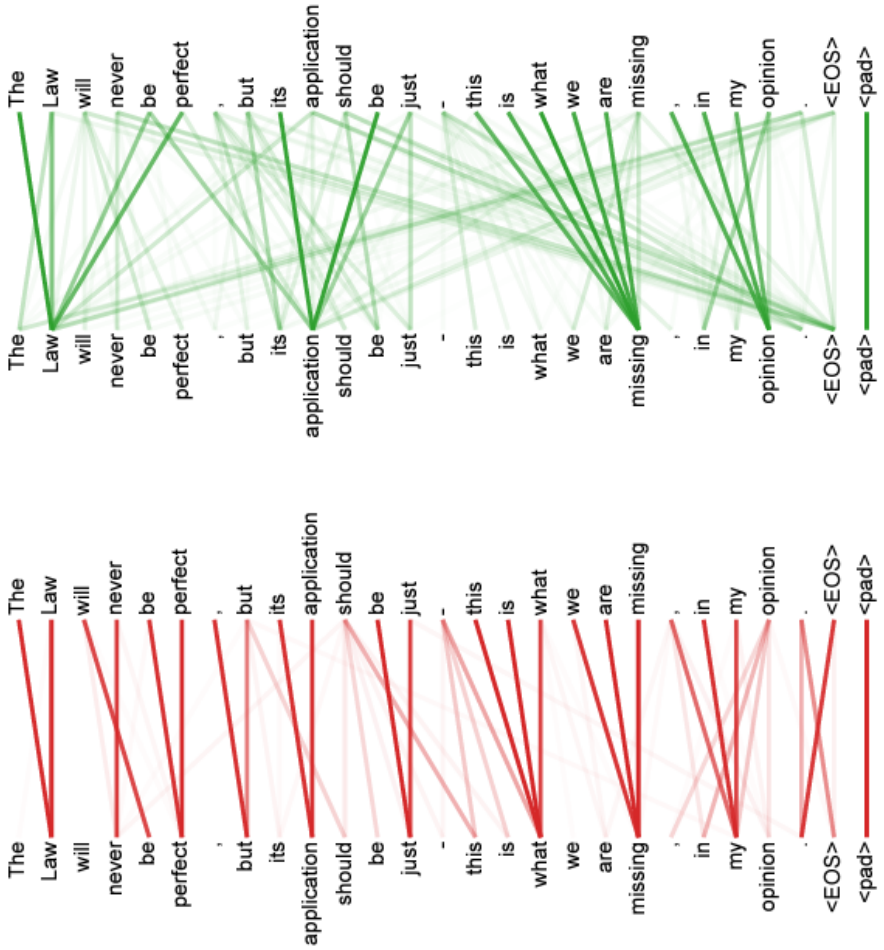
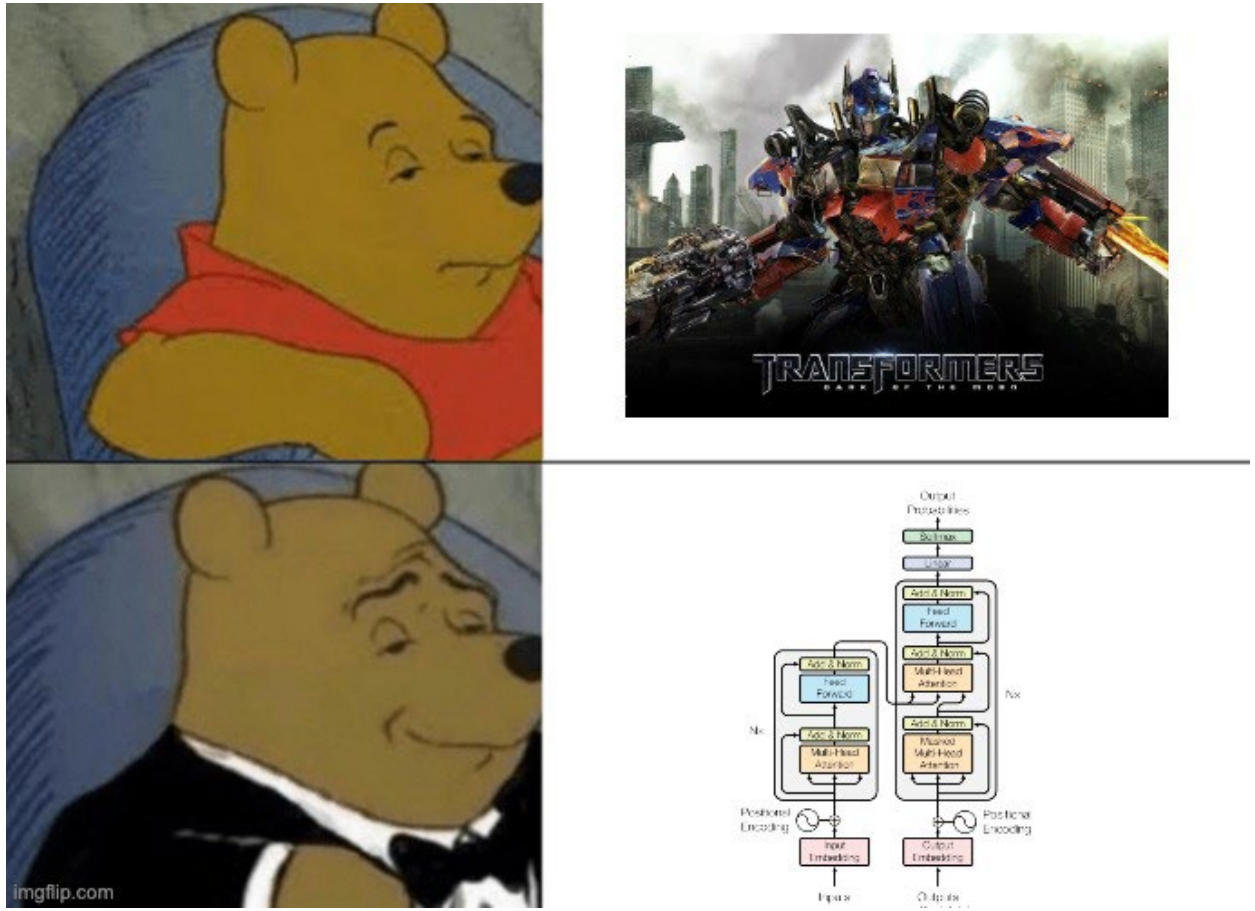


Figure 5: Many of the attention heads exhibit behaviour that seems related to the structure of the sentence. We give two such examples above, from two different heads from the encoder self-attention at layer 5 of 6. The heads clearly learned to perform different tasks.

[Attention Is All You Need, Vaswani 2017]

Coding time 2: HuggingFace Transformers library

Conclusion meme